

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 5

Operating Systems

The xv6 Shell from the Outside

FAF-241, Ciur Alexandrina

Checked by:
Reitman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 The tree and the file types **2**
1.1 Commands and output 2
1.2 Answers 3

2 Inodes and links **3**
2.1 Commands and output 3
2.2 Answers 5

3 Permissions **6**
3.1 Commands and output 6
3.2 Answers 7

4 Disks and the kernel as files **8**
4.1 Commands and output 8
4.2 Answers 9

What I did

I worked as a normal user (not root) in the directory `~/os-lab5` on a Linux virtual machine (VirtualBox, user `vboxuser`, host `lux11`). I ran the commands from the lab sheet and observed the file tree, inodes and links, permissions, and disks. All terminal output is included below as text.

```
vboxuser@lux11:~$ whoami
vboxuser
vboxuser@lux11:~$ mkdir -p ~/os-lab5 && cd ~/os-lab5
vboxuser@lux11:~/os-lab5$ touch report.md
```

1 The tree and the file types

1.1 Commands and output

```
$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var

$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 12 12:51 /dev/null
drwxr-xr-x 141 root root 12288 Oct 12 08:46 /etc
-rw-r--r--   1 root root  2778 Oct  5 14:03 /etc/passwd
drwxrwxrwt  15 root root   340 Oct 12 12:59 /tmp

$ ls -l /dev | head
total 0
crw-r--r-- 1 root     root     10, 235 Oct 12 12:51 autofs
drwxr-xr-x 2 root     root        500 Oct 12 12:51 block
drwxr-xr-x 2 root     root         80 Oct 12 12:51 bsg
crw------- 1 root     root     10, 234 Oct 12 12:51 btrfs-control
drwxr-xr-x 3 root     root         60 Oct 12 12:51 bus
lrwxrwxrwx+1 root     root          3 Oct 12 12:51 cdrom -> sr0
drwxr-xr-x 2 root     root       3820 Oct 12 12:51 char
crw------- 1 root     tty       5,   1 Oct 12 12:51 console
lrwxrwxrwx 1 root     root         11 Oct 12 12:51 core -> /proc/kcore

$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin

$ du -sh /var/log 2>/dev/null
36M     /var/log

$ du -sh /usr 2>/dev/null
5.1G    /usr
```

```
$ ls -l /dev | grep ^b | head -5
brw-rw---- 1 root     disk      7,   0 Oct 12 12:51 loop0
brw-rw---- 1 root     disk      7,   1 Oct 12 12:51 loop1
brw-rw---- 1 root     disk      7,  10 Oct 12 12:51 loop10
brw-rw---- 1 root     disk      7,  11 Oct 12 12:51 loop11
brw-rw---- 1 root     disk      7,  12 Oct 12 12:51 loop12
```

1.2 Answers

(1) Type of each path. The type is given by the first letter of the `ls -l` output.

Path	First letter	Type
`/etc`	`d`	directory
`/etc/passwd`	`-`	regular file
`/bin`	`l`	symbolic link
`/dev/null`	`c`	character device
`/tmp`	`d`	directory (with the sticky bit `t`)

(2) What is `/bin`? It is a **symbolic link** to `usr/bin` (`/bin -> usr/bin`), confirmed by the `file` command. On this system `/bin` is not a separate directory: the "merged `/usr`" layout is used. Also, `/usr/bin/ls` is itself a symbolic link to `../lib/cargo/bin/coreutils/ls`, so this system uses the Rust implementation of coreutils.

(3) Character and block devices in `/dev`.

- Character device: `/dev/null` (`crw-rw-rw-`, major 1, minor 3). Others seen in the output: `console`, `autofs`, `btrfs-control`.
- Block device: `/dev/loop0` (`brw-rw--`, major 7, minor 0). The `loop*` devices are used to mount files as disks (here, snap packages). The real disk, `/dev/sda`, appears in Part 4.

(4) Which is bigger, `/var/log` or `/usr`? `/usr` is 5.1 GB and `/var/log` is 36 MB, so `/usr` is about 145 times larger ($5.1 \times 1024 \approx 5222$ MB, and $5222/36 \approx 145$). `/usr` holds all installed programs and libraries, while `/var/log` holds only log files.

2 Inodes and links

2.1 Commands and output

An inode, and a name that is not part of it.

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 268073      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/vboxuser)   Gid: ( 1000/vboxuser)
```

```
Access: 2026-10-12 13:25:29.248860050 +0000
Modify: 2026-10-12 13:25:28.492850079 +0000
Change: 2026-10-12 13:25:28.492850079 +0000
 Birth: 2026-10-12 13:25:28.491850066 +0000

$ ls -li notes.txt
268073 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 12 13:25 notes.txt
```

Hard link, then remove the "original".

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
268073 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 12 13:25 hard.txt
268073 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 12 13:25 notes.txt

$ rm notes.txt
$ cat hard.txt ; ls -li hard.txt
hello
268073 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 12 13:25 hard.txt
```

Symbolic link, then remove its target.

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
268073 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 12 13:25 hard.txt
269296 lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 12 13:32 soft.txt -> hard.txt

$ cat soft.txt
hello

$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 12 13:32 soft.txt -> hard.txt
```

Directories have links too.

```
$ mkdir d ; ls -ld d
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 12 13:34 d

$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 vboxuser vboxuser 4096 Oct 12 13:34 d
```

Size is not space.

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 vboxuser vboxuser 2 Oct 12 13:35 one.txt
4.0K    one.txt

$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 vboxuser vboxuser 1.0G Oct 12 13:35 big.img
0       big.img

$ rm big.img
```

2.2 Answers

(1) Inode and number of names. In `stat`, the field `Inode` is the inode number (here `268073`); in `ls -li` it is the first column. The field `Links` (the second column of `ls -l`) tells how many names (directory entries) the file has: 1 at the start, 2 after `ln`, and back to 1 after `rm notes.txt`.

(2) Why `hard.txt` was still readable. `notes.txt` and `hard.txt` were two names for the *same* inode (`268073`), which is why the link count was 2. The name is not part of the inode: it is only an entry in the directory that points to the inode. `rm` removes a name (a directory entry) and decrements the link count; the data and the inode are freed only when the count reaches 0 and no process has the file open. After `rm notes.txt` the count was 1, so the file and its content (`hello`) were still there under the name `hard.txt`.

(3) Why `cat soft.txt` failed. A symbolic link is a separate file, with its own inode (`269296`, different from `268073`), that stores only the *path* of the target. After `rm hard.txt` the last name of the target and the target itself were gone, so the path stored in `soft.txt` leads nowhere (a dangling link) and `cat` reports `No such file or directory`. The link itself still exists, which `ls -l` shows. Its size is **8 bytes**, because it stores the path `hard.txt`, which has exactly 8 characters (h-a-r-d-.-t-x-t).

(4) Link count of a directory. A new empty directory has **2** links: (a) its name `d` in the parent directory, and (b) its own entry `.` inside it, which points to itself. Each subdirectory contains an entry `..` that points back to its parent. After creating `d/x` and `d/y`, two more `..` entries point to `d`, so the count becomes $2 + 2 = \mathbf{4}$.

(5) Why 2 bytes use 4.0K and 1 GB uses 0. The file system allocates space in whole blocks, not bytes; the block size here is 4096 bytes (`IO Block: 4096` in `stat`). The file `one.txt` contains 2 bytes (`x` and a newline), but it occupies one full block, so `du` reports 4.0K. The file `big.img` is a *sparse* file: `truncate -s 1G` only sets the logical size to 1 GB and allocates no data blocks. The `ls -lh` command shows the logical size (1.0G) and `du` shows the space actually used (0). So the size of a file is not the same as the space it takes on disk.

3 Permissions

3.1 Commands and output

Files.

```
$ echo secret > s.txt
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 vboxuser vboxuser 7 Oct 12 13:37 s.txt

$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

$ chmod u+r s.txt ; cat s.txt
secret

$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 vboxuser vboxuser    0 Oct 12 13:37 new.txt
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 12 13:37 newdir
```

Directories.

```
$ mkdir box ; echo hi > box/f
$ chmod u=r,go= box  ; ls box ; cat box/f     # read only
f
cat: box/f: Permission denied

$ chmod u=x,go= box  ; ls box ; cat box/f     # execute only
ls: cannot open directory 'box': Permission denied
hi

$ chmod 755 box
```

Special bits and other people's files.

```
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 16 root root 360 Oct 12 13:34 /tmp

$ cat /etc/shadow
cat: /etc/shadow: Permission denied

$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1314 Oct  5 14:13 /etc/shadow
```

3.2 Answers

(1) Octal modes as `rwx` strings. Each octal digit is, from left to right, the permissions of the owner, the group and the others ($r = 4$, $w = 2$, $x = 1$).

Octal	String	Meaning	Real use
600	`rw----`	owner reads and writes; nobody else	private files, e.g. SSH private key `~/.ssh/id_rsa`
644	`rw-r-r-`	owner writes; everybody reads	ordinary files, e.g. `/etc/passwd`, web pages
755	`rwxr-xr-x`	owner has full access; others read and execute	programs and shared directories, e.g. `/usr/bin/ls`
700	`rwx---`	only the owner has access	private directories, e.g. `~/.ssh`

(2) Why ownership does not help with `chmod 000`. When a process opens a file, the kernel checks the permission bits of the *class* that applies to it. The user is the owner, so the owner bits are used, and with mode 000 they are `--`: reading is denied (`Permission denied`). Ownership gives no automatic access to the contents. What ownership does give is the right to *change the mode*: the owner could run `chmod u+r s.txt` and read the file afterwards. Only root bypasses these checks. (Note that `chmod u+r` on mode 000 gives 400, so `ls -l` shows `-r----`.)

(3) The umask. My umask is `0002`. It removes bits from the default mode: files start from 666 and directories from 777. For `new.txt`: $666 \,\&\, \sim 002 = 664$, i.e. `rw-rw-r-`. For `newdir`: $777 \,\&\, \sim 002 = 775$, i.e. `rwxrwxr-x`. This matches the output of `ls -ld`. The umask `0002` only removes the write bit for others and is typical on Ubuntu, where every user has a private group.

(4) The `box` experiment.

- With **read only** (`r-`): `ls box` works and prints `f`, because `r` allows *listing the names* in the directory. But `cat box/f` fails with `Permission denied`: without `x` the directory cannot be *traversed*, so the file inside cannot be reached.
- With **execute only** (`-x`): `ls box` fails, since the names cannot be listed without `r`. But `cat box/f` works and prints `hi`, because `x` allows *accessing* an entry inside the directory when its name is known.

In short, on a directory `r` means "list the names" and `x` means "pass through it / access entries by name".

(5) The `s` and the `t` bits.

- `-rwsr-xr-x` on `/usr/bin/passwd`: the `s` in the owner's execute position is the **setuid** bit. The program runs with the privileges of the file's *owner* (root), not of the user who started it. `passwd` needs this because it has to write the new password hash into `/etc/shadow`, which is `-rw-r---` owned by root: a normal user can neither read nor write it (as shown by `cat /etc/shadow` giving `Permission denied`). Thanks to setuid, a user can change only his own password, in a controlled way, through the program.

- `drwxrwxrwt` on `/tmp`: the `t` is the **sticky bit**. `/tmp` is writable by everybody, but with the sticky bit a file can be deleted or renamed only by its owner (or by the directory owner or root). So users cannot delete each other's temporary files.

4 Disks and the kernel as files

4.1 Commands and output

```
$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/1643
loop2    7:2    0  66.8M  1 loop /snap/core24/2124
loop3    7:3    0    20M  1 loop /snap/desktop-security-center/151
loop4    7:4    0  14.2M  1 loop /snap/desktop-security-center/206
loop5    7:5    0 260.3M  1 loop /snap/firefox/8763
loop6    7:6    0  16.5M  1 loop /snap/firmware-updater/226
loop7    7:7    0  12.7M  1 loop /snap/firmware-updater/262
loop8    7:8    0 614.5M  1 loop /snap/gnome-46-2404/164
loop9    7:9    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop10   7:10   0   1.5M  1 loop /snap/hwctl/123
loop11   7:11   0   1.5M  1 loop /snap/hwctl/131
loop12   7:12   0   402M  1 loop /snap/mesa-2404/1839
loop13   7:13   0  18.8M  1 loop /snap/prompting-client/222
loop14   7:14   0  18.4M  1 loop /snap/prompting-client/228
loop15   7:15   0  11.8M  1 loop /snap/snap-store/1390
loop16   7:16   0  50.1M  1 loop /snap/snapd/27710
loop17   7:17   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0    25G  0 disk
|-sda1   8:1    0     1M  0 part
`-sda2   8:2    0    25G  0 part /
sr0     11:0    1  1024M  1 rom

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G  2.4M  1.6G   1% /run
/dev/sda2        25G  8.0G   16G  35% /
tmpfs           4.0G     0  4.0G   0% /dev/shm
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           4.0G  8.0K  4.0G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           804M   84K  803M   1% /run/user/1000

$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)

$ grep -c processor /proc/cpuinfo
6

$ head -3 /proc/meminfo
```

```
MemTotal:        8223120 kB
MemFree:         1833564 kB
MemAvailable:    5340052 kB

$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   9855
Ngid:   0

$ ls -l /proc/self/fd
total 0
lrwx------ 1 vboxuser vboxuser 64 Oct 12 13:43 0 -> /dev/pts/0
lrwx------ 1 vboxuser vboxuser 64 Oct 12 13:43 1 -> /dev/pts/0
lrwx------ 1 vboxuser vboxuser 64 Oct 12 13:43 2 -> /dev/pts/0
lr-x------ 1 vboxuser vboxuser 64 Oct 12 13:43 3 -> /proc/9870/fd
```

4.2 Answers

(1) My disk. The system has one real disk, `sda` (25 GB), with two partitions. The `loop*` devices are snap packages mounted from files, and `sr0` is the virtual CD drive.

```
sda  (disk, 25G)
|-- sda1  (1M)   : no mount point (tiny partition, typically for boot)
`-- sda2  (25G)  : mounted at  /   , file system type: ext4
```

The partition mounted at `/` is `/dev/sda2`, with the file system **ext4** (`rw,relatime`), according to `mount`.

(2) How full is the root file system? `/dev/sda2` has 25 GB, of which 8.0 GB are used and 16 GB are available: **35%** used.

(3) CPU cores and RAM. `/proc/cpuinfo` lists 6 `processor` entries, so the VM has **6 CPU cores**. `/proc/meminfo` shows `MemTotal: 8223120 kB`, which is 8223120/1024/1024 ≈ 7.84 GiB, so about **8 GB of RAM** (the memory given to the virtual machine).

(4) Which program is named in `/proc/self/status`, and why? It shows `Name: head`. `/proc/self` is a special link that always points to the process that *accesses* it. The file was opened and read by the `head` command, so "self" is the process of `head`. This is also why `ls -l /proc/self/fd` showed the descriptors of the `ls` process itself: 0, 1 and 2 are the terminal `/dev/pts/0` (standard input, output and error), and 3 is the directory being read. Files in `/proc` are not stored on disk: they are generated by the kernel on demand, which shows that in Linux the kernel state is also exposed as files.

Conclusion

In this lab I checked on a real Linux system the main ideas of the file system. I saw that everything lives in one tree, and that the first letter of `ls -l` gives the file type: even devices (`/dev/null`, `/dev/loop0`) and links (`/bin`) are files. With `stat` and `ls -li` I saw that a name is only a directory entry pointing to an inode: a hard link is another name for the same inode, so the file survives `rm` until its link count is 0, while a symbolic link is a separate file with its own inode that stores a path and breaks when the target is removed. I also saw that the size of a file is not the space it occupies (4 KB blocks, sparse files). In the permission experiments I learned how `rwx` works for files and directories, how the umask (`0002`) determines the default modes, and why the setuid bit (`passwd`) and the sticky bit (`/tmp`) exist. Finally, `lsblk`, `df` and `/proc` showed my disk (`sda`, with `sda2` as ext4 at `/`, 35% full), the CPU (6 cores) and the RAM (about 8 GB), and that the kernel exposes its own state as files.